

课程笔记

Codex 通用工具设计：Subagents 与 Apply_Patch

五道口纳什 & AI

2026 年 3 月 28 日

Codex General Tool sub-agents

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 35 分钟

视频链接：未填写

目录

1 引言：拆开 Coding Agent 的黑盒	2
2 Tool 规格的三层架构	2
2.1 第一层：内部规格 (Tool Spec)	2
2.2 第二层：Wire Layer (网络传输层)	3
2.3 第三层：执行层 (Handlers)	3
2.4 本章小结	3
3 Apply Patch：结构化的代码编辑	3
3.1 为什么不直接重写文件？	3
3.2 为什么模型能理解 Lark 语法？	4
3.3 Patch 的执行流程	4
3.4 本章小结	4
4 Execute Commands 与 Write Stdin	4
4.1 Execute Commands	4
4.2 Write Stdin	4
4.3 本章小结	5
5 Subagent 系统设计	5
5.1 为什么需要 Subagent？	5
5.2 五个 Subagent 操作	5
5.3 Agent Nickname 机制	5
5.4 Spawn Agent 的返回值与上下文感知	6
5.5 Wait Agent 的语义	6
5.6 Subagent 完成后的通知机制	6
5.7 本章小结	6
6 View Image：多模态 Agent 的桥接工具	6
6.1 为什么需要单独的 Tool？	6
6.2 工作流程	7
6.3 去重优化	7
6.4 本章小结	7
7 请求体的落盘与调试	7
7.1 如何查看完整请求体	7
7.2 本章小结	7
8 总结与延伸	8
8.1 拓展阅读	8

1 引言：拆开 Coding Agent 的黑盒

本期继续探索 Codex，从前几期的 Context Engineering、Runtime、Harness 角度之后，本期聚焦于 Codex 内部 **12 个预定义工具** (Tools) 的设计。讲者反复强调的核心理念是：**没有魔法，千万不要把 Agent 当黑盒**——要去 Review 和 Guide Agent 的内部处理过程，尤其当模型陷入死循环或表现不符合预期时。

Codex 的 12 个预定义 Tool

1. Apply Patch (编辑代码文件)
2. Execute Commands (执行命令)
3. Write Stdin (标准输入写入)
4. Update Plan (更新计划)
5. Request User Input (请求用户输入)
6. Web Search (网络搜索)
7. View Image (查看图像)
8. Spawn Agents (创建子 Agent)
9. Send Inputs (向子 Agent 发送指令)
10. Wait (等待子 Agent)
11. Close Agents (关闭子 Agent)
12. Resume (恢复子 Agent)

2 Tool 规格的三层架构

Codex 源码（主要在 `spec.rs` 文件中）对工具定义了三层架构：

2.1 第一层：内部规格 (Tool Spec)

Codex 自己维护的抽象定义，包含：

- **name**: 工具名称。
- **description**: 工具描述。
- **parameters**: 入参定义。
- **output_schema**: 输出结构描述。

output_schema 不会暴露给 API

内部规格中定义了 output_schema (描述工具的返回值结构), 但这个字段**不会**出现在发给远端 API 的请求体中。模型对输出结构的感知, 完全依赖于 runtime 将 function call output 回填到上下文中。

2.2 第二层: Wire Layer (网络传输层)

“Wire”即变现/网线,指工具在网络传输中的定义。Codex 通过 create_tools_json_for_response_api 函数将内部 Tool Spec 序列化为 JSON, 塞入 response.create 请求体的 tools 字段。

Function Calling 的设计特点

标准的 Function Calling 协议中, Tool 定义**只有入参描述, 没有出参描述**。这意味着模型在调用前并不“知道”返回值的精确格式——它通过 runtime 回填的 function call output 来学习返回结构。

2.3 第三层: 执行层 (Handlers)

当模型产生一个 Function Call 后, Codex Runtime 进入 Handlers:

1. 解析参数并做校验。
2. 在**本地**执行操作 (执行不在远端服务器上发生)。
3. 将结果封装为 Function Call Output, 放入上下文的消息列表中。

2.4 本章小结

三层架构实现了关注点分离: 内部规格定义完整的工具语义, Wire Layer 负责序列化传输, Handlers 负责本地执行与结果回填。

3 Apply Patch: 结构化的代码编辑

3.1 为什么不直接重写文件?

Apply Patch 是 Codex 中最核心的代码编辑工具。它的设计理念是: **不让模型自由地重写整个文件, 而是输出结构化的 diff**——添加了什么行、删除了什么行、添加了什么文件、删除了什么文件。

Patch DSL: 一种受限的领域特定语言

Apply Patch 定义了一套基于 Lark 语法的 DSL (Domain Specific Language):

- 使用 `*** begin patch / *** end patch` 包裹。
- 用 `*** update file / *** add file / *** delete file` 标记操作。
- 强制模型以 diff 形式表达编辑意图, 而非随意改文本。

3.2 为什么模型能理解 Lark 语法？

1. 在请求体中，Codex 将 Lark Grammar 的文本作为工具描述的一部分发送给模型。
2. 在第一条 System Instruction 中，也包含了 Apply Patch 的使用说明。
3. 指令明确要求：**对单个文件的编辑必须使用 Apply Patch，不要用 Python 读写文件**；当简单的 Shell 命令或 Apply Patch 可以满足时，不要另辟蹊径。

3.3 Patch 的执行流程

Handler 收到 Patch 内容后并不立即修改文件，而是：

1. **验证语法**：检查 Patch 是否符合 Lark Grammar。
2. **推导影响范围**：确定改了哪些文件、需要哪些读写权限。
3. **权限审批**：如有额外审批要求则先获取许可。
4. **执行 Patch**：应用 diff 到文件系统。

3.4 本章小结

Apply Patch 将”改代码”这一高风险操作约束为结构化的 diff DSL，既节省上下文（不需要输出整个文件），又让每次编辑可审计、可验证。

4 Execute Commands 与 Write Stdin

4.1 Execute Commands

Execute Commands 是一次性的命令调用：启动进程、运行一段、获得输出。它同时兼容”执行模型生成的代码脚本”——模型可以先生成一段代码，再通过 Execute Commands 运行。

”Patch is All You Need”

讲者引用了业界流行的说法：将应用程序变成命令行可交互的形式后，所有操作都可以通过 Patch（编辑）和 Command（执行）两个原语完成。

4.2 Write Stdin

Write Stdin 用于向**正在运行**的程序持续发送输入——即与交互式 CLI（如 iPython、Node REPL、MySQL、GDB 等）进行对话，发送输入并获取增量输出。

Execute Commands vs Write Stdin

- **Execute Commands**：一次性调用。启动 → 运行 → 获取输出 → 结束。
- **Write Stdin**：持续交互。向已运行的进程发送新指令，获取增量响应。

4.3 本章小结

Execute Commands 覆盖了绝大多数”运行某段代码/命令”的需求，Write Stdin 则补充了交互式会话的场景，两者共同构成了 Agent 执行能力的基础。

5 Subagent 系统设计

5.1 为什么需要 Subagent ?

三大动机

1. **隔离上下文**: 避免所有信息塞入单个 Master Agent 的 Context Window 导致溢出。
2. **并发执行**: 多个独立任务可以由不同 Subagent 同时处理。
3. **更长的交付物**: Master Agent 将繁重工作委派给 Subagent, 自己只做最终的 Reduce/汇总。

5.2 五个 Subagent 操作

Codex 的 Subagent 系统由五个 Function Tool 组成:

1. **Spawn Agents**: 创建子 Agent。
2. **Send Inputs**: 向已有子 Agent 发送新指令。
3. **Wait**: 阻塞等待一个或多个子 Agent 达到最终状态。
4. **Close Agents**: 显式关闭/下档子 Agent。
5. **Resume**: 恢复一个已关闭或退出的子 Agent, 沿其原有上下文继续工作。

Subagent 的通信方向是单向的

子 Agent 只在**结束时**才产生输出。当前的结构设计中, 子 Agent 没有主动与主 Agent 交互的模式——所有通信都是主 Agent 向子 Agent 发送。

5.3 Agent Nickname 机制

Codex Runtime 维护了一个 Agent Nicknames 池, 主要是历史科学家和学者的名字。创建子 Agent 时:

1. 优先查看配置文件中是否自定义了 nickname。
2. 否则从内置名字池中选取一个未被占用的名字。
3. Nickname 用于前端展示, 所有内部交互使用 Agent ID。

5.4 Spawn Agent 的返回值与上下文感知

模型如何感知创建的 Subagent ?

Function Calling 的 Tool 定义中没有返回值描述。那么模型如何知道自己创建了哪个 Sub-agent?

答案在于: **Spawn Agent 的返回值** (包含 agent_id 和 nickname) 作为 Function Call Output 被 Runtime 回填到模型上下文中。模型在后续推理时自然能从上下文中读取到这些信息, 从而建立起”我创建了哪个 Agent、它的 ID 是什么”的关联。

5.5 Wait Agent 的语义

- Wait 不是 sleep, 而是对 Agent 状态流的订阅——更像是 join / await 操作。
- 它会阻塞 Master Agent 的后续推理, 直到目标 Subagent 达到最终状态。
- 有超时机制 (最长约一小时)。
- **何时使用**: 仅当 Master Agent 的下一步必须依赖 Subagent 的结果时才触发。如果自己还有别的事情可做, 不要急于 Wait。

Spawn Agent 的触发条件

Codex 对 Spawn Agent 的使用设置了两条强指令:

1. 只有用户**显式要求** Subagent 分派或并发执行时, 才创建 Subagent。
2. 例外: 如果是对 Codebase 做**深度、全面的 Research 或系统分析**, 不需要用户许可即可创建 Subagent。

因此, 想测试 Subagent 流程, 需要在 Query 中显式提到”用 subagent 并发完成”。

5.6 Subagent 完成后的通知机制

如果 Master Agent 没有调用 Wait, Subagent 完成后并不会”失联”。Runtime 会在后台自动将 Subagent 的完成消息以 **User** 身份注入到 Master Agent 的上下文中, 确保信息不丢失。

5.7 本章小结

Subagent 系统通过上下文隔离和并发执行, 突破了单一 Context Window 的限制。五个操作原语 (Spawn / Send Inputs / Wait / Close / Resume) 构成了完整的子 Agent 生命周期管理。

6 View Image: 多模态 Agent 的桥接工具

6.1 为什么需要单独的 Tool ?

GPT 是多模态模型, 理论上可以直接处理 Base64 编码的图片。但在 Agent 系统中, 需要一个专门的 Tool 来实现:

Agent 自主决定”看什么图”

View Image 的意义在于: **Agent 需要自主决定何时查看哪张图片**。模型通过调用 View Image Tool, 告诉 Runtime ”我现在需要看这张图”, Runtime 负责读取文件、编码为 Base64、作为 Function Call Output 注入上下文。

这一步 (Prepare Image Input) 必须由 Runtime 完成, 而非预先塞入所有图片。

6.2 工作流程

1. 用户提供一个图片的绝对路径 (如”请读取这张照片: /path/to/image.jpg”)。
2. 模型产生 Function Call: `view_image(file_path="/path/to/image.jpg")`。
3. Runtime 读取文件、转换为 Base64。
4. 以 Image Content 形式注入到后续的 `response.create` 输入中。
5. 模型基于视觉理解继续推理。

6.3 去重优化

Tool Description 中包含一条规则: **如果这张图片在历史上已经看过, 则不需要再次读取**——可以直接复用之前上下文中的视觉信息。

6.4 本章小结

View Image 是本地图像与多模态模型之间的桥接工具, 将”看图”这一操作显式化为 Function Call, 使 Agent 能够按需获取视觉信息。

7 请求体的落盘与调试

7.1 如何查看完整请求体

GPT 的 `response.create` API 默认不落盘请求体。但 Codex 提供了调试选项:

```
1 export CODEX_LOG_REQUESTS=1
2 # 请求体将保存到 ~/.codex/logs-1/skill-lite
```

Listing 1: 启用请求体落盘的环境变量

落盘的 JSON 文件包含完整的请求体: instructions、developer 消息、user 消息、12 个 Tool 定义 (含 Apply Patch 的 Lark Grammar) 等。

7.2 本章小结

通过环境变量启用落盘, 可以完整审视 Codex 发送给 API 的请求体, 是调试和理解 Agent 行为的重要手段。

8 总结与延伸

本期从 Tool Design 的角度深入拆解了 Codex 的 12 个预定义工具，核心要点回顾：

1. **三层架构**：内部规格 (Tool Spec) → Wire Layer (网络传输) → Handlers (本地执行)。
2. **Apply Patch**：用 Lark Grammar 约束的 Diff DSL，将代码编辑结构化、可审计化。
3. **Execute Commands + Write Stdin**：覆盖一次性执行和交互式会话两种场景。
4. **Subagent 系统**：通过 Spawn / Send Inputs / Wait / Close / Resume 五个原语管理子 Agent 生命周期，实现上下文隔离与并发执行。
5. **View Image**：多模态 Agent 的图像桥接工具，将”看图”显式化为 Function Call。
6. **Function Call Output 的回填**：模型对工具返回值的感知完全依赖 Runtime 将结果注入上下文，而非 Tool 定义中的 output_schema。

理解了这 12 个 Tool 的设计，加上前几期对 Context Engineering 的分析，就能对各种 Coding Agent 的内部运作形成深入直观的认知。核心启示是：**Agent 的能力边界 = 底层模型能力 + Agent Runtime 维护的工具与上下文 + Harness 层的流程约束。**

8.1 拓展阅读

- Codex 开源代码仓库 (spec.rs、handlers/ 目录)
- OpenAI Function Calling 文档
- Lark Parser 语法定义
- Multi-Agent 系统设计模式